

Article

An SQL Query Description Problem with AI Assistance for an SQL Programming Learning Assistant System

Ni Wayan Wardani ^{1,2} , Nobuo Funabiki ^{1,*}, Htoo Htoo Sandi Kyaw ¹, Zihao Zhu ¹, I Nyoman Darma Kotama ^{1,2} , Putu Sugiartawan ^{1,2} and I Nyoman Agus Suarya Putra ³

¹ Graduate School of Environmental, Life, Natural Science and Technology, Okayama University, Okayama 700-8530, Japan; pj5w1e4c@s.okayama-u.ac.jp (N.W.W.); htoohtoo@okayama-u.ac.jp (H.H.S.K.); pjco5fyn@s.okayama-u.ac.jp (Z.Z.); p18z9yov@s.okayama-u.ac.jp (P.S.)

² Faculty of Technology and Information, Indonesian Institute of Business and Technology, Denpasar 80225, Indonesia

³ Faculty of Business and Creative Design, Indonesian Institute of Business and Technology, Denpasar 80225, Indonesia; nyomansuarya@instiki.ac.id

* Correspondence: funabiki@okayama-u.ac.jp

Abstract

Today, relational databases are widely used in information systems. SQL (structured query language) is taught extensively in universities and professional schools across the globe as a programming language for its data management and accesses. Previously, we have studied a web-based programming learning assistant system (PLAS) to help novice students learn popular programming languages by themselves through solving various types of exercises. For SQL programming, we have implemented the grammar-concept understanding problem (GUP) and the comment insertion problem (CIP) for its initial studies. In this paper, we propose an SQL Query Description Problem (SDP) as a new exercise type for describing the SQL query to a specified request in a MySQL database system. To reduce teachers' preparation workloads, we integrate a generative AI-assisted SQL query generator to automatically generate a new SDP instance with a given dataset. An SDP instance consists of a table, a set of questions and corresponding queries. Answer correctness is determined by enhanced string matching against an answer module that includes multiple semantically equivalent canonical queries. For evaluation, we generated 11 SDP instances on basic topics using the generator, where we found that Gemini 3.0 Pro exhibited higher pedagogical consistency compared to ChatGPT-5.0, achieving perfect scores in Sensibleness, Topicality, and Readiness metrics. Then, we assigned the generated instances to 32 undergraduate students at the Indonesian Institute of Business and Technology (INSTIKI). The results showed an average correct answer rate of 95.2% and a mean SUS score of 78, which demonstrates strong initial student performance and system acceptance.

Keywords: database programming; SQL query description problem (SDP); self-study; programming learning assistant system (PLAS); generative AI



Academic Editors: Razi Iqbal and Shereen Ismail

Received: 13 November 2025

Revised: 30 December 2025

Accepted: 31 December 2025

Published: 9 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Today, relational databases play a central roles in information systems by managing large volumes of data efficiently. Database programming using Structured Query Language (SQL) has become a core skill for data scientists and software developers [1]. A lot of universities and professional schools offer courses related to SQL in their curricula, teaching

data definitions, retrievals, and manipulations [2–6]. However, despite its importance, database programming is not offered in many universities, and novice students often struggle to write correct SQL queries because of their complexity, making both syntactic and semantic mistakes that hinder learning progress [7–9].

The difficulties faced by novice students can be effectively addressed through the lens of Cognitive Load Theory (CLT). Novice learners often struggle with high intrinsic cognitive load stemming from the inherent complexity of SQL logic, which is frequently compounded by the extraneous cognitive load associated with configuring and managing complex database environments. To facilitate effective learning, it is crucial to minimize these extraneous demands and manage intrinsic difficulty through scaffolding, thereby freeing up working memory resources for germane load—the active construction of knowledge schemas. Guided by these principles, this study aims to optimize the learning process by providing a streamlined, web-based environment and structurally sequenced exercises.

To address these cognitive load challenges in SQL learning, we have developed a web-based Programming Learning Assistant System (PLAS) to support self-study of popular programming languages such as Java, Python, and JavaScript. PLAS is designed and implemented to help novice students study programming by themselves through solving various types of exercises. Among them, we implemented the grammar-concept understanding problem (GUP) and the comment insertion problem (CIP) for learning keyword meaning and basic syntax of SQL programming as its initial studies. These exercises may help students in initial recognition and grammar understanding. However, they do not provide realistic practices in composing SQL queries for database tables.

To provide realistic practice, instructors encounter challenges when designing varied SQL programming exercises. This task typically entails constructing diverse database schemas and crafting matching queries, which is time-consuming and often repetitive [10]. Recent studies show that automatic schema generation by generative AI such as ChatGPT-3.5 can produce reasonable cores but often inconsistent relationships and redundancy, indicating the need for careful validation [10]. Thus, teachers need tools that can reduce manual works while ensuring schema correctness and pedagogical values.

In this paper, we propose an SQL Query Description Problem (SDP) as a new exercise type inside PLAS. An SDP instance contains a database table schema and a set of questions that require students to write SQL queries for data retrieval or manipulation with their correct answers. Student answers are evaluated automatically using enhanced string matching against a bank of canonical solutions that includes multiple semantically equivalent variants. To reduce preparation efforts and increase variety, we also propose a generative AI-assisted SQL query generator that combines large human-labelled schema collections called Spider dataset [11] with popular generative AI models such as ChatGPT 5.0 [12] and Gemini 3.0 Pro [13] to produce candidate schemas, questions, and reference SQL queries to the dataset.

For evaluation, we generated 11 SDP instances on fundamental SQL topics using the generator with the two generative AI models. The correctness of them was manually validated by teachers before use. We found that ChatGPT-5.0 demonstrated superior technical efficiency with fewer execution errors and a lower mean error rate, making it highly effective for rapid query generation. However, Gemini 3.0 Pro emerged as the more consistent model for pedagogical purposes, achieving perfect scores in Sensibleness, Topicality, and Readiness with absolute inter-rater consensus ($\kappa = 1.00$). While Gemini 3.0 Pro required slightly more manual syntax debugging than ChatGPT-5.0, its superior instructional alignment and logical stability make it more suitable for the standardized requirements of this study. ($\kappa = 1.00$). While Gemini 3.0 Pro required slightly more

manual syntax debugging than ChatGPT-5.0, its superior instructional alignment and logical stability make it more suitable for the standardized requirements of this study.

Then, we assigned the SDP instances to 32 undergraduate students at the Indonesian Institute of Business and Technology (INSTIKI) during a 120-min class. Our analysis of student performances showed that the average correct answer rate was 95.2% and the average number of submissions was 1.56 per question. The usability test using a questionnaire yielded 78 (Grade B) for the average SUS score. These results indicate that the proposed SDP with the AI-assisted generator using Gemini 3.0 Pro can be a reliable solution for SQL programming studies by novice students.

1.1. Contributions of This Study

To clarify the novelty of this work, the unique contributions of this study compared to prior research in the PLAS ecosystem and existing SQL learning tools are as follows:

1. **Introduction of SDP:** We propose the SQL Query Description Problem (SDP) as a new exercise type that focuses on logical–semantic mapping, bridging the gap between natural language requirements and SQL syntax.
2. **LLM-based Automation:** We develop a structured prompting framework that enables the automated generation of diverse SQL exercises, significantly reducing manual effort for instructors compared to previous manual methods used in GUP and CIP.
3. **Enhanced Assessment Methodology:** We introduce an enhanced string-matching logic for the automated evaluation of student answers. This methodology accounts for multiple semantically equivalent canonical solutions and normalizes syntactic variations, ensuring a robust and reliable assessment without the need for manual grading.
4. **Comparative Generative AI Evaluation:** We provide a detailed comparative evaluation of Gemini 3.0 Pro and ChatGPT-5.0 specifically for generating educational SQL content, using metrics that ensure pedagogical suitability.
5. **Educational Feasibility:** We demonstrate the practical utility of the system through an empirical study involving 32 students, confirming system acceptance and task-solving performance.

1.2. Paper Organization

The remaining sections of this paper are organized as follows: Section 2 overviews related works in literature. Section 3 details the main software and datasets used. Section 4 provides an overview of the existing PLAS system. Section 5 presents the SQL Query Description Problem (SDP) for database programming. Section 6 describes the methodology and the experimental design. Section 7 evaluates the proposal. Section 8 discusses the findings, implications, and limitations. Section 9 concludes this paper with future works.

2. Related Works

In this section, we introduce works in the literature related to this study.

2.1. SQL Programming Learning

First, we introduce works on SQL programming learning.

In [3], Garner et al. introduced SQL in Steps (SiS) as an online environment that combines a graphical user interface with a textual representation for improving SQL learning. SiS simplifies the transition from graphical to textual interfaces, making the transition seamless. Their study on the application to first-year undergraduates confirmed its potential.

In [14], Mitrovic et al. presented Intelligent Teaching System (ITS) research that develops electronic tutors that mimic one-on-one instruction with a human tutor. Their SQL-

Tutor is designed as a practice environment, assuming students have completed database management courses. Although it only covers SQL SELECT statements, the system's significance can be extended to other SQL commands and relational database languages, as queries pose significant challenges for students.

In [15], Akhuseyinoglu et al. introduced Database Query Analyzer (DBQA) as a learning tool that uses interactive data visualizations to demonstrate effects of clauses and conditions on SQL SELECT statements. This tool provides result sets similar to those maintained by a database management system during query processing. DBQA modifies the result set according to each clause and condition, highlighting the clause currently being processed.

Overall, research on SQL learning shows that a variety of tools and systems have been developed to improve students' comprehension and proficiency in learning SQL. They include interactive visualizations that show the effects of clauses in queries and graphical interfaces that make the transition easier.

2.2. Generative AI for Programming Education

Second, we introduce works on use of generative AI models for programming education.

In [16], Luckin et al. showed that by building a proper AI model, educational contents can be made more engaging and effective to accomplish goals of individualized education.

In [17], Chen et al. analysed the impact of AI on education, verifying that AI has been widely used in education, helping teachers work more effectively, and improving the overall quality of teaching.

In [18], Baidoo-Anu explored potential benefits and limitations of ChatGPT in enhancing teaching and learning. They provided recommendations on how ChatGPT can be used to maximize its impacts on teaching and learning.

In [19], Choi et al. showed that GPT-5 represents a significant advancement in educational language models, addressing shortcomings such as hallucinations and pedagogical alignment. This approach offers new opportunities across various subjects and learning profiles. Despite limited empirical studies, the evidence indicates that GPT-5 aligns with learning objectives and adapts to diverse educational needs.

Based on the research mentioned above, it is clear that generative AI models have significant impacts on various educational needs, such as helping teachers work more effectively in making and presenting learning content, thereby improving the quality of teaching.

2.3. Generative AI for Query Generation

Third, we introduce works on use of generative AI model for query generations.

In [20], Pornphol et al. verified the relational completeness of SQL query codes generated by ChatGPT as one of the most widely used generative AI models. ChatGPT generated relational algebra, relational calculus, and SQL statements for five natural language test questions.

In [21], Li et al. presented a method for automatic test case development that initially creates a database and subsequently employs a generative AI model to ascertain the ground truth, defined as the anticipated execution outcomes of the corresponding SQL query on this database.

Together, these studies underscore the potential of generative AI models to streamline query generations and validations, improving efficiency and reliability in database management systems while addressing challenges in automation and accuracy.

2.4. Reducing Teacher Workloads

Fourth, we introduce works on use of generative AI model for reducing teacher workloads.

In [22], Gupta et al. investigated impacts of generative AI models on teacher productivity in higher education, highlighting their potential to save time and reduce workloads. They revealed that generative AI models streamline administrative tasks, enhance teaching efficiency, and optimize assessment processes. This research contributes to the discourse on their roles in sustainable productivity improvements.

In [23], Hashem et al. explored the use of ChatGPT in secondary schools to reduce teacher burnouts. They focused on the creation of materials and lesson plans, which are crucial factors causing burnouts. This study uses customized questions for science, math, and English classes, evaluating ChatGPT abilities in individualized planning and content creations. It emphasized benefits of task-specific suggestions and AI-human collaborations for personalized planning, aligning with UAE's AI integration goals.

Both studies highlighted significant roles of generative AI models in reducing teacher workloads and improving productivity. Together, these studies showed how they can effectively support teachers, enabling them to focus more on teaching and less on time-consuming administrative tasks.

2.5. Cognitive Load Theory in Programming Learning

In [24], Mason et al. redesigned an introductory database course using Cognitive Load Theory (CLT) to address high failure rates. By re-sequencing content, utilizing segmentation, and replacing visual tools with raw SQL to reduce extraneous load, they significantly improved student outcomes. Notably, final exam scores increased and the failure rate dropped from 42% to 8%.

In [25], Yousoof et al. proposed a framework to mitigate working memory limitations in programming education. By employing concept maps that mirror long-term memory schemas, alongside part-code methods and information filtering, the approach segments information effectively. This visualization strategy significantly reduces cognitive load and split attention for novice learners.

While both studies validate CLT in database and programming contexts, there remains a paucity of research regarding its application in AI-assisted SQL programming environments. Building upon these findings, this study extends the CLT framework to SQL query formulation, specifically aiming to reduce cognitive load through the proposed SDP, which utilizes generative AI to provide scaffolded practice and immediate feedback.

2.6. Formative Assessment Theory

Ma et al. [26] utilize correct answer rates as a formative indicator to assess knowledge mastery. By visualizing these rates at the class level, they identify common misconceptions and provide targeted feedback to enable students to recognize weaknesses and improve performance.

Similarly, Sudo et al. [27] employ correct answer rates to monitor learner understanding and evaluate instructional interventions. Their findings confirm that quantitative performance indicators are effective in supporting learning diagnosis and driving instructional improvement.

Harvey and Aggarwal [28] investigate submission timing as a behavioral indicator in programming education. They found that early submissions significantly correlate with higher exam scores, suggesting that submission time is a meaningful predictor of student engagement and learning outcomes.

In the specific context of SQL learning, Ma et al. [26] further interpret correct answer rates through CLT. They posit that low correct rates indicate high cognitive burden, while improvements suggest efficient schema construction, thereby guiding the reduction of extraneous cognitive load.

In conclusion, these studies collectively demonstrate that quantitative metrics—ranging from accuracy rates to submission timing—serve as vital diagnostic tools. By leveraging these indicators, teachers can effectively monitor cognitive load, identify behavioral patterns, and provide timely interventions to support student mastery.

3. Software and Dataset

In this section, we describe the main software tools and datasets used in the proposal.

3.1. Generative AI

Generative Artificial Intelligence (AI) models, particularly those based on the Transformer architecture introduced in 2017 [29], have demonstrated exceptional capabilities in natural language processing and content creation. This presents a significant opportunity for automation and AI-powered content creation in education, which is leveraged in this study to reduce the workload of teachers. The LLM models used in this study are as follows:

OpenAI's GPT-5: Launched in August 2025, this model offers advanced language understanding, multi-level reasoning, and programming support [12]. A related model, ChatGPT-3.5, has shown an 87% accuracy rate in generating and correcting SQL syntax [30].

Google's Gemini 3.0 Pro: Released in November 2025, this multimodal generative AI model is known for its strong reasoning, comprehension, and coding capabilities, capable of handling complex problems and supporting contexts up to one million tokens [13]. A previous version, Gemini 1.0 Pro, has shown an 80% accuracy rate in SQL syntax generation [30].

3.2. Spider Dataset

The Spider dataset is a major breakthrough in the fields of natural language processing (NLP) and text-to-SQL system developments, designed to handle complex, cross-domain semantic parsing tasks. A team in Department of Computer Science at Yale University created the Spider dataset, which includes 10,181 natural language questions and 5693 unique SQL queries. They are carefully labelled by 11 undergraduate students, covering 200 databases with many tables from 138 different domains like education, aviation, and entertainment [11]. Spider dataset is unique because it uses different databases for training and testing, which allows a generative AI model to adapt to new structures and questions, unlike older datasets like ATIS or WikiSQL that usually focus on matching patterns within one area. With a state-of-the-art model accuracy of only 12.4% in the “split database” setting, Spider offers a new challenge that emphasizes the importance of true semantic understanding, rather than rote memorization, and is publicly available at <https://yale-lily.github.io/spider> (accessed on 5 August 2025) to support [11].

From 200 databases with diverse domains, we selected college_2 database schema. The database schema for college_2 consists of 11 tables and approximately 30,000 data entries. The domain is familiar for teachers and students, which is relevant to daily life and educational environments [31]. Teachers can utilize this dataset to illustrate practical applications of SQL in educational data analysis, while students can learn to interpret realistic database schemas and complete challenging tasks, thereby enhancing their understanding of SQL programming and data processing. The schema of college_2 can be found in Appendix B.

4. Review of Programming Learning Assistant System

In this section, we briefly review our programming learning assistant system (PLAS) and its application to learning in our previous studies.

4.1. System Overview

PLAS is a self-learning web-based platform that supports well-known programming languages like C, Java, and Python. For teachers, it supports generating instances and analysing students' responses. For students, it supports practicing and using the system. An overview of the software architecture in PLAS is shown in Figure 1.

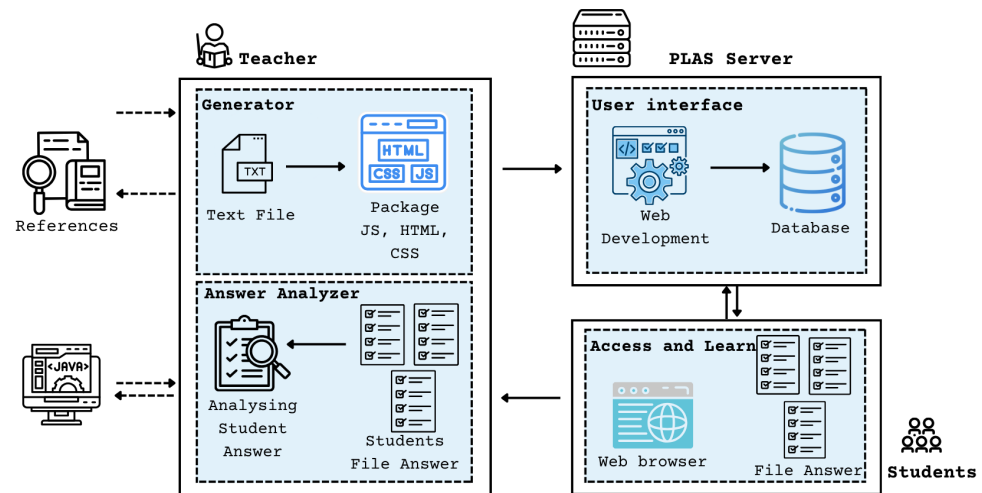


Figure 1. System overview of PLAS.

4.2. Exercise Problems

PLAS provides a range of programming exercise tasks with diverse difficulty levels and objectives. The Element fill-in-blank problem (EFP) requires students to complete absent elements in given source code [32]. The Grammar-concept understanding problem (GUP) requires students to respond to inquiries regarding the grammar within source code or libraries [33]. The Value trace problem (VTP) requires students to ascertain the values of essential variables or anticipated results [34].

In our preliminary applications of PLAS to database programming learning, the GUP was adopted to understand grammatical concepts, the Comment Insertion Problem (CIP) was used to understand the structures and logics of source code, and the SQL Query Description Problem (SDP) was used to understand basic syntax of SQL database programming for novice students to perform queries ranging from simple to complex. This step-by-step learning approach allows students to systematically improve their fundamental database programming competencies.

4.3. Technical Implementation of PLAS

The PLAS platform was designed as a web-based self-directed learning environment accessible via a web browser. The interface was implemented utilizing HTML5, CSS, and JavaScript to facilitate access to the exercises, allowing fast responses to inputs for immediate feedback. User interaction and data storage capabilities are executed on a browser with JavaScript. This design streamlines deployment and maintenance.

The exercises can be developed and managed by teachers through an administrative interface that enables them to establish question instances, categorize them, and provide correct answers along with validation criteria. Upon answer submission by a student, the system automatically compares them with the stored correct answers, identifies the incorrect ones, and documents them for future review.

Students engage with the PLAS platform using a web browser to complete the exercises. The platform stores the related data, including the submission and answer histories, which will be utilized to assess student progress and address specific challenges.

4.4. Instance Generation Functions

The instance generator has been implemented for each exercise type and language within PLAS. It operates together with the settings for different types of exercise problems. Figure 2 shows how it processes and masks the answers for EFP.

In this process, teachers first input a source code according to the topic. Then, the generator masks the correct answers and generates the corresponding HTML, CSS, and JavaScript files before presenting them to students. During exercise executions, student inputs are evaluated through string matching against the correct answers stored in the program. The same mechanism is applied to generating instances for other exercise types. On the student side, the masked parts in EFP are displayed as blank input fields, allowing learners to fill in their answers directly through the web interface.

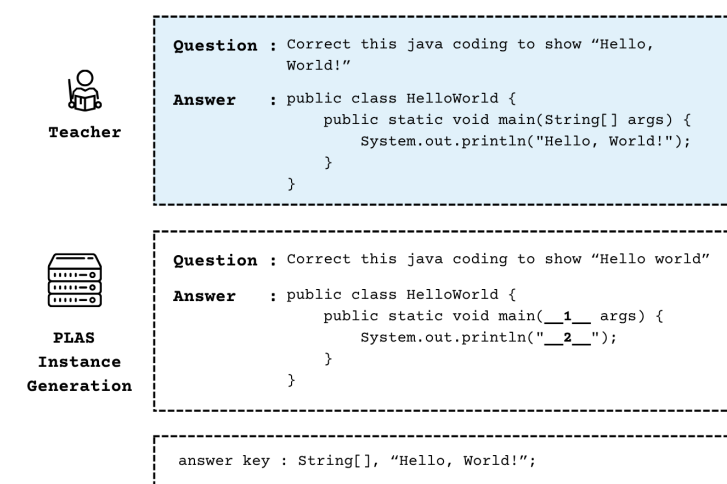


Figure 2. Example of EFP instance generation.

5. Proposal of SQL Query Description Problem

In this section, we present the SQL Query Description Problem (SDP) with a generative AI-assisted SQL query generator.

5.1. Overview

SDP with the generator is designed to help teachers generate practical SQL exercises efficiently and allow students to practice query writing interactively. Figure 3 illustrates the workflow of the proposed SDP.

First, teachers use the generator to automatically create new SDP instances. Teachers begin by selecting a learning topic such as SELECT, UPDATE, or JOIN and choosing a preferred *generative AI model* such as ChatGPT 5.0 or Gemini 3.0 Pro. Based on these inputs, the generator generates pairs of questions and their corresponding SQL queries by applying prompt engineering techniques to the model.

The generated question–answer pairs should be checked by teachers for syntactic correctness and query running ability within a MySQL environment. Only questions that pass this verification can be included for further reviews. Teachers then perform manual validations to ensure that the generated contents align with the course materials and the intended difficulty levels. Once validated, these contents are exported into SDP instance files, ready to be uploaded into PLAS.

In this paper, we extended the existing instance generator of PLAS to support the SQL-specific structure in SDP, using the available SQL keywords listed in Appendix A as a reference. Each validated SDP instance consists of a database schema, multiple question

statements, and their correct query answers. Students access these SDP exercises through the PLAS user interface on a browser.

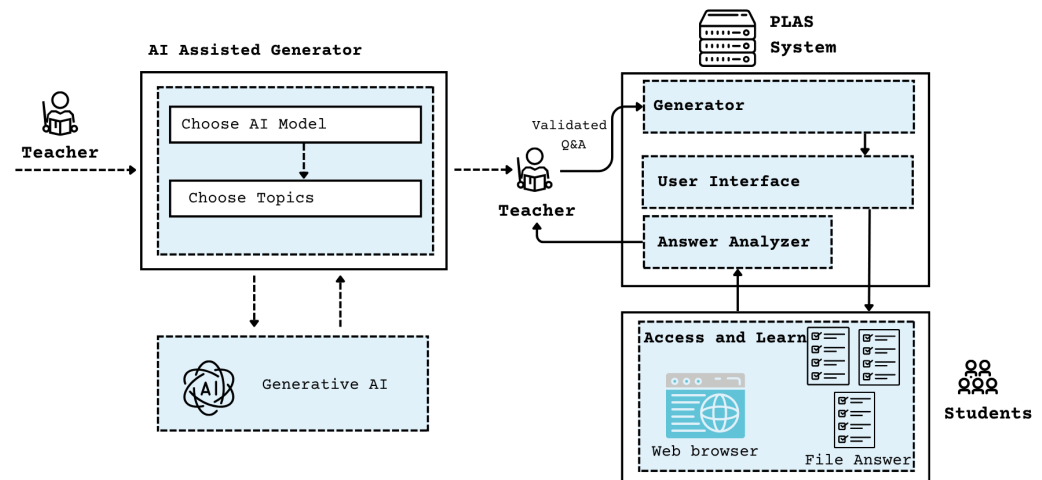


Figure 3. Overview of proposal.

5.2. Generative AI-Assisted SQL Query Generator

The generative AI-assisted SQL query generator is developed to support teachers in producing diverse and practical SDP instances. It leverages generative AI models to automatically generate question–answer pairs based on real database schemas. Teachers can use this generator through a simple graphical interface, as shown in Figure 4. They need to select a desired SQL topic such as SELECT, UPDATE, or JOIN and specifying the generative AI model to use. The generator then creates a corresponding set of SQL exercises that consist of question statements and correct SQL queries.

Once teachers start the generator, it prepares prompts as shown in Table 1, including the schema information and SQL topic instructions. The generated results of question–answer pairs are automatically checked in syntax correctness and running ability on the target MySQL database system. Only the pairs that are executed successfully are saved for teacher validations. Then, teachers should review, edit, or discard generated contents before uploading them to PLAS. This workflow allows for efficient and semi-automated content creations while maintaining human oversights on accuracy and pedagogical consistency.

5.2.1. Question-Answer Pair Generation

The question–answer pair generation process relies on Large Language Models (LLMs), specifically ChatGPT 5.0 and Gemini 3.0 Pro, chosen for their strong performances in text generations and structured query formulations. These models can interpret natural language prompts and produce consistent SQL query statements that follow the MySQL syntax rules.

For the input context, we use Spider dataset as a well-known human-labelled dataset for cross-domain SQL generations developed by Yale University [11]. This dataset provides database schemas and query examples across multiple domains, allowing generative AI models to generate realistic questions and accurate queries at various difficulty levels.

Table 1 illustrates the design of a standardized prompt structure used to ensure clarity and output consistency. The prompt instructs the selected generative AI model to produce enhanced question–answer pairs, where each question is accompanied by multiple semantically equivalent SQL queries and explanations. This standardized prompting ensures that each generated content includes a mix of basic and intermediate SQL operations, promoting balanced difficulty and comprehensive learning coverage.

SQL Query Generator

Choose AI Model
ChatGPT-5.0

Database Schema
College 2 Schema

SQL Topic
DELETE

GENERATE

Copy All Queries

Schema: College 2 Schema
Topic: DELETE

10 DELETE Practice Exercises — College 2 Schema

Tables: students, enrollments, courses, instructors, departments
These exercises focus on safe and practical uses of DELETE with WHERE clauses, joins, and subqueries.
Always practice DELETE in a transaction or on test data!

Easy 1. Delete a specific student by ID (e.g., student_id = 999)

```
DELETE FROM students
WHERE student_id = 999;

-- Alternative (using EXISTS)
DELETE FROM students
WHERE student_id = 999 AND EXISTS (
  SELECT 1 FROM students s WHERE s.student_id = 999
);
```

Figure 4. User Interface of Generative AI-assisted SQL query generator interface.

Table 1. Prompt template for generative AI model to generate SQL question–answer pairs.

Prompt Template:
“Create 10 SQL questions based on the attached database schema, focusing on [topic], with two levels of difficulty. For each question, provide an explanation and multiple alternative query formulations that are semantically equivalent and yield the same results.”

5.2.2. Generated Results

Table 2 presents examples of generated content. In this work, we redefine the traditional concept of a “question–answer pair” to accommodate the flexibility of SQL syntax. Instead of a single reference answer, each generated instance functions as a question–solution set, consisting of one natural language question and multiple semantically equivalent valid queries.

These examples demonstrate the system’s capability to produce clear, executable SQL queries aligned with educational topics.

Teachers then perform manual validations to confirm that the generated pairs aligned with course objectives and the corresponding schema design. Only validated pairs are subsequently registered as SDP instances in PLAS, where students can access to them for self-learning and practice.

Table 2. Examples of generated question–answer pairs with alternative queries.

Level	Question	Answer (SQL Query)
Easy	How can you retrieve the names of all departments from the department table?	Option 1: <code>SELECT dept_name FROM department;</code> Option 2: <code>SELECT d.dept_name FROM department AS d;</code>
Medium	Write a query to get the names of students (student.name) and the courses they are taking (course.title). Use the student and takes tables.	Option 1: <code>SELECT s.name, c.title FROM student s JOIN takes t ON s.ID = t.ID JOIN course c ON t.course_id = c.course_id;</code> Option 2: <code>SELECT s.name, c.title FROM student s, takes t, course c WHERE s.ID = t.ID AND t.course_id = c.course_id;</code>

5.3. SDP Instance Generation Program

The answer instance generator for PLAS was modified to support the structure and logic of SQL Query Description Problem (SDP), to create new SDP instances that align with SQL query constructions using predefined keywords and question templates. A list of 31 keywords and their corresponding questions is shown in Appendix A. Each keyword corresponds to a specific concept in SQL database programming and follows the rules of Data Manipulation Language (DML) and data retrieval operations.

The answer interface generation process ensures that the created instances reflect the real syntax and operations of SQL. Table 3 illustrates how a DML operation, such as INSERT, is used to build an SDP instance. A teacher selects the keyword, defines its question, and registers both in PLAS as a new instance.

Table 3. Example of DML-based SDP keyword and question pair.

Category	Keyword	Question Example
Data Manipulation Language (DML)	INSERT	Write an SQL command to add a new record into the student table with the values ('S05', 'Lisa', 'Physics', 3).

In addition to DML, the generated SDP instances also include exercises for data retrieval. They focus on querying and analysing information using commands such as JOIN, ORDER BY, and GROUP BY. Table 4 shows some of them. These additional ones are made so that students will practice writing SQL queries that cover both data manipulation and complex retrieval logic, reinforcing their understanding of query syntax and data relationships.

Table 4. Examples of keyword and question pairs for data retrieval.

Category	Keyword	Question Example
Data Retrieval	JOIN	Write a query to display student names and their enrolled course titles by joining the student and takes tables using the student ID.
Data Retrieval	ORDER BY	Retrieve all student names and grades from the student table and sort the result in descending order by grade.

5.3.1. SDP Instance Generation Procedure

The generation of a new SDP instance running on a web browser involves several steps. The process begins with teachers selecting relevant learning materials from SQL database programming classes, textbooks, or online resources such as [35,36]. These materials serve as the foundation for designing exercises that students can later study and practice.

From the materials, teachers need to identify important SQL keywords and choose the corresponding question templates listed in Appendix A. The relevant questions are then selected from the question–answer pairs generated by the generative AI-assisted SQL query generator. Each selected question–answer pair is manually reviewed by teachers to ensure the correctness, clarity, and alignment with the learning objectives.

At this step, any redundant or overlapping question–answer pairs are discarded to maintain variety within the exercise set. After validation, the finalized content is compiled into an SDP instance text file that contains all relevant data, including schema information, images, questions, and correct answers.

Finally, the answer interface generator automatically produces the necessary HTML, CSS, and JavaScript files from the SDP instance text file [37,38] that can run on a web browser. This integration allows students to access and practice the exercises interactively, with automatic grading and feedback mechanisms built in PLAS.

5.3.2. Instance List Page

The instance list page allows students to view and manage the assigned SDP instances. Figure 5 shows the layout of this page, where each SDP instance is displayed with its title and progress status. The instances highlighted in green indicate that all the questions in them have been answered correctly by a student, while those in yellow show partial completions. The instances with no highlighting are those that have not yet been attempted. Students are advised to read provided instructions before attempting exercises. This visual feedback mechanism helps learners track their progress and identify which topics require further review.

5.3.3. Answer Page

The *answer page* enables students to input and test SQL queries for questions interactively. Figure 6 illustrates the page providing a text input area where students can write the SQL query in response to each question. Upon submission, the JavaScript program on the page checks the answer using string matching and executes the query to confirm correctness.

Incorrect submissions are highlighted in red, while correct ones appear in white. Students may resubmit their answers until all questions are correctly solved. Each attempt is recorded, including the submission time, the answers, and results, allowing teachers to monitor learning progress and common errors.

5.4. SDP Assessment Logic

Our SDP assessment system employs a string matching mechanism to validate student submissions. This process compares the student's query against a comprehensive set of alternative answers—representing semantically equivalent variants—stored within the answer module. This approach allows the system to accept various correct syntactic forms for a single problem.

Guidance

[View Guidance](#)

Problems

Problem No	Problem Name	Remark
1	SQL Select	
2	SQL Update	
3	SQL Insert	
4	SQL Delete	
5	SQL Join	
6	SQL Character Function	
7	SQL Numeric Function	
8	SQL Date Function	
9	SQL Aggregate Function	
10	SQL Order By	
11	SQL Group By and Having	

Submission

1

2

Your Record

3

Figure 5. Screenshot of the instance list page displaying SDP instances and progress status.

Problem #1

SQL Query Description Problem

SQL Select

a. Write a query to select all columns from the student table.

Your SQL Query:

b. Write a query to select the ID, name, and salary from the instructor table.

Your SQL Query:

c. Write a query to select all columns from the student table where the tot_cred is greater than 100.

Your SQL Query:

Figure 6. Implementation view of the SDP answer page interface.

6. Evaluation Setup

In this section, we present a methodology and experimental design.

6.1. Generative AI-Assisted SQL Query Generator Evaluation Setup

To ensure the validity and reliability of the proposed generative AI-assisted SQL query generator, an evaluation framework has been established, encompassing the system configuration and testing scenarios as detailed below.

6.1.1. Evaluation Metrics

To assess the quality of problems generated by a generative AI, we used metrics adopted from Sarsa et al. [39], as detailed in Table 5.

Table 5. Evaluation metrics for assessing the quality of SQL question–answer pairs generated by generative AI, adapted from Sarsa et al.

Aspect	Question	Description
A. Subjective Quality Metrics (Assessed by Experts using a 5-Point Likert Scale)		
Sensibleness	Does the problem description describe a sensible problem?	Assesses the clarity, grammatical accuracy, and factual correctness of the description. (1: Highly Illogical/5: Highly Logical)
Novelty	Are we unable to find the SQL exercise via online search (Google) of the problem statement?	Assesses the degree of uniqueness of the SQL query problem/description. (1: Already Existing/5: Highly Novel)
Topicality: concept/keyword	Does the answer to the problem statement require the primed concept/keyword?	Assesses the relevance of the description to the requested SQL concepts/keywords. (1: Not Relevant/5: Highly Relevant)
Topicality: extras	Does the generated problem statement and solution use the extra primed words (e.g., chain-of-thought, etc.)?	Assesses the quality of integration of additional primed elements into the output. (1: Failure/5: Perfect)
Readiness: answer	Is the answer to the generated problem provided?	Assesses the completeness and sufficiency of the description as a learning aid for students. (1: Not Ready/5: Highly Ready)
B. Objective Metric (Numerical)		
Readiness: runnability	How many corrections do we need to make to be able to run the solution query correctly?	Integer

6.1.2. Assessment Procedure

The evaluation is conducted independently by two (2) lecturers of the Database course each having a minimum of three years of teaching experience at the university level. These assessors serve as expert validators. They are provided with a detailed list of output descriptions and assessment instructions (including the operational definitions above) without being informed whether the descriptions were generated by AI or manually created, to minimize bias.

6.1.3. Inter-Rater Reliability

Inter-Rater Reliability (IRR) is a crucial statistical measure designed to assess the degree of consistency and agreement among the ratings provided by two or more independent assessors [40]. In this study, the evaluation was conducted independently by two expert validators—Database course lecturers with a minimum of three years of teaching experience. Employing IRR is essential to confirm that the evaluation of the output descriptions is not

reliant on the subjective preferences of a single individual, but represents an objective and reliable consensus among experts.

6.1.4. Bias Minimization Procedure

To ensure the collected IRR data accurately reflects genuine agreement, strict measures were taken to minimize potential bias. The expert validators performed their evaluation tasks independently of one another. Crucially, they were blinded to the source of the output descriptions—whether they were AI-generated or manually created—thus eliminating the risk of rater expectancy bias. This procedure ensures that the assessment focuses purely on the quality of the description based on the defined operational metrics.

6.1.5. Cohen’s Kappa Coefficient

The Cohen’s Kappa (κ) coefficient was selected as the appropriate statistical measure for assessing IRR in this study. Unlike Fleiss’ Kappa, which is used for multiple raters, Cohen’s Kappa is specifically designed to measure the agreement between two independent raters assigning categorical or ordinal ratings to a set of items [41].

Given that the evaluation utilized a 5-point Likert scale, a Weighted Cohen’s Kappa was employed. This variation accounts for the magnitude of disagreement between raters (e.g., a difference between a score of 4 and 5 is penalized less than a difference between 1 and 5), which is essential for ordinal data. The coefficient is calculated using the following formula:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

where P_o represents the observed proportion of agreement, and P_e denotes the expected proportion of agreement by chance. Based on the interpretation scale by Landis and Koch [42], the global IRR for this study achieved a linear weighted kappa of 0.631, indicating a substantial agreement between the two human raters across all learning topics.

6.1.6. Interpretation of Kappa Value

The resulting Cohen’s Kappa (κ) value is interpreted to determine the strength of agreement between the two human raters. This study adopts the widely accepted interpretation benchmarks proposed by Landis and Koch [42], as summarized in Table 6.

A satisfactory level of agreement is typically indicated by a κ value of at least 0.61, corresponding to a substantial level of agreement. In this evaluation, the global weighted kappa value was 0.631. This result meets the threshold for substantial agreement, supporting the conclusion that the metrics assessments of the AI models’ outputs demonstrate high reliability and reflect a credible and objective consensus between the raters.

6.2. SDP PLAS Evaluation Setup

For evaluations, we generated 11 SDP instances with 67 questions. Table 7 lists the topic and the number of questions for each instance. Then, we assigned them to 32 first-year undergraduate students who were taking the SQL database programming course at Indonesian Institute of Business and Technology (INSTIKI), Indonesia. Instances with IDs 1 through 5 were included in the midterm exam, whereas instances with IDs 6 through 11 are reserved for the final exam. The duration time of midterm exam was 60 min, which was sufficient for students to repeat answer submissions until they got the correct answers.

Table 6. Interpretation of Kappa Values [42].

Kappa Value (κ)	Strength of Agreement
<0.00	Poor
0.00–0.20	Slight
0.21–0.40	Fair
0.41–0.60	Moderate
0.61–0.80	Substantial
0.81–1.00	Almost Perfect

Table 7. Generated SDP instances with corresponding SQL topics.

No.	SQL Topic	Number of Questions
1	SELECT Statement	5
2	UPDATE Statement	5
3	INSERT Statement	8
4	DELETE Statement	6
5	JOIN Operation	5
6	Character Functions	9
7	Numeric Functions	8
8	Date Functions	5
9	Aggregate Functions	6
10	ORDER BY Clause	8
11	GROUP BY and HAVING Clauses	4

6.2.1. Participants and Demographic Profile

The experiment involved 32 undergraduate students enrolled in the “Introduction to Database” course (typically a 1st or 2nd-semester course) at the Indonesian Institute of Business and Technology (INSTIKI) [43]. All participants were novice learners of SQL, as the experiment was conducted immediately following the introductory lectures on basic SQL commands such as SELECT, FROM, WHERE, and JOIN.

Demographic information for the participants was as follows: 81.3% were male (26 participants) and 18.7% were female (6 participants). All participants had similar academic backgrounds (Information Technology major) and had no prior professional experience with SQL or relational databases. The homogeneity of the sample (novice level, same major) was intentional to control for prior knowledge variables and focus on the feasibility of the SDP for beginners.

The homogeneity was chosen to mitigate confounding variables related to prior SQL expertise, ensuring that the measured learning outcomes were attributable to the SDP method rather than varying skill levels. As discussed in Section 8.5, this intentional restriction limits the external validity, but strengthens the internal validity of the findings regarding the efficacy of SDP for introductory SQL students.

6.2.2. Experimental Procedure

The experiment was conducted during a scheduled 120-min laboratory session. The procedure adhered to the following steps:

- Step 1: **Preparation (10 min)** : Participants were briefed on the purpose of the study and the ethical consent form (including anonymity and voluntary participation).
- Step 2: **Introduction to SDP (10 min)**: Participants were given a brief tutorial on the concept of the SDP and how to interact with the web-based PLAS interface.

- Step 3: **Task Execution (60 min):** Each participant was instructed to solve a predetermined set of 5 SDP instances (IDs 1–5), comprising a total of 29 questions, using the proposed learning assistant system. Participants were permitted to solve the questions in any order.
- Step 4: **Data Collection (40 min):** After completing the tasks, participants were asked to fill out two standardized instruments:
- A survey questionnaire to evaluate system feasibility (correctness rate and submission times);
 - The SUS questionnaire to assess the system’s usefulness and usability.

All data collection was automated by the PLAS, and the experiment was supervised by one of the co-authors (affiliated with INSTIKI) and a dedicated teaching assistant to ensure standardized conditions across all participants.

6.2.3. System Usability Scale

Following the assignments, a System Usability Scale (SUS) questionnaire was distributed to the participating students. Originally developed by John Brooke in 1986 [44], the SUS has been known as a simple yet reliable method for measuring the usability of interactive systems. Table 8 presents the 10 SUS questions in this evaluation, each rated on a five-point Likert scale, where 1 represents “Strongly Disagree” and 5 represents “Strongly Agree”.

Each answer is scored by a point between 1 and 5. For odd-numbered questions, one point is subtracted from the participant’s response, while for even-numbered ones, the response value is subtracted from five points. The adjusted values are then summed and multiplied by 2.5 to obtain a final score between 0 and 100.

For example, if a participant rates question 1 with 4, the adjusted value becomes $(4 - 1) = 3$. If question 2 is rated with 3, the adjusted value becomes $(5 - 3) = 2$. These adjusted points are added to the overall score.

According to Table 9, a score between 0 and 50.9 is considered “Not Acceptable”, a score between 51 and 70.9 is “Marginal”, and a score ranging from 71 to 100 is “Acceptable”. Table 10 shows the percentile-based grade interpretation, where a score of 80.3 or higher corresponds to Grade A, and a score below 51 corresponds to Grade E.

Table 8. SUS questions.

No.	Question
1	I think that I would like to use this system frequently.
2	I found the system unnecessarily complex.
3	I thought the system was easy to use.
4	I think that I would need the support of a technical person to be able to use this system.
5	I found the various functions in this system were well integrated.
6	I thought there was too much inconsistency in this system.
7	I would imagine that most people would learn to use this system very quickly.
8	I found the system very cumbersome to use.
9	I felt very confident using the system.
10	I needed to learn a lot of things before I could get going with this system.

Table 9. Acceptability SUS ranges for user satisfaction levels.

SUS Score Range	Interpretation
0–50.9	Not acceptable
51–70.9	Marginal
71–100	Acceptable

Table 10. SUS percentile rank interpretation.

Grade	Score Range
A	Score ≥ 80.3
B	$74 \leq \text{Score} < 80.3$
C	$68 \leq \text{Score} < 74$
D	$51 \leq \text{Score} < 68$
E	Score < 51

7. Evaluation Results

In this section, we present a comprehensive evaluation of the proposal, focusing on the AI model’s performance and its feasibility for the self-study of SQL query by novice students.

7.1. Generative AI-Assisted SQL Query Generator Evaluation Results

Our evaluation focuses on six key metrics. They include Sensibleness, Novelty, Topicality: Concept, Topicality: Extras, Readiness: Answer, and Readiness: Runnability. These indicators measure the logical accuracy, creativity, topical relevance, and technical correctness of the generated SQL exercises.

7.1.1. Inter-Rater Reliability Results

To ensure the objectivity and reliability of the evaluation process, an IRR test was conducted using Cohen’s Kappa (κ). This analysis measures the level of agreement between two expert raters for each evaluation metric across a total sample of 220 assessments (110 problems generated by each AI model). The results of the IRR analysis are summarized in Table 11.

Table 11. Global Inter-Rater Reliability Results ($n = 220$).

Metric	Cohen’s Kappa (κ)	Interpretation
Sensibleness	0.86	Almost Perfect Agreement
Novelty	0.91	Almost Perfect Agreement
Topicality: Concept	1.00	Almost Perfect Agreement
Topicality: Extras	0.85	Almost Perfect Agreement
Readiness: Answer	0.87	Almost Perfect Agreement
Runnability (Objective)	0.72	Substantial Agreement

The IRR results, as summarized in Table 11, confirm a robust level of consistency between the two expert raters. Based on the Table 6, five out of six metrics achieved an “Almost Perfect Agreement” with Kappa coefficients (κ) ranging from 0.85 to 1.00.

The Topicality: Concept metric achieved a perfect Kappa score ($\kappa = 1.00$), indicating complete agreement between the raters regarding the alignment of the generated problems with the intended SQL concepts. The Novelty metric also demonstrated exceptionally high consistency, with Kappa values of 0.91. These results confirm a highly synchronized rater perception concerning the originality of the generated SQL queries. Metrics reflecting

subjective stability, including Sensibleness ($\kappa = 0.86$), Topicality: Extras ($\kappa = 0.85$), and Readiness: Answer ($\kappa = 0.87$), similarly maintained strong levels of agreement.

Interestingly, the Runnability metric yielded a Kappa score of 0.7, which is categorized as “Substantial Agreement”. While still indicating a strong level of consensus, this lower coefficient compared to other metrics suggests that identifying technical SQL errors—such as subtle syntax mistakes or logical flaws—presented more room for interpretative differences between the raters. Nevertheless, the overall IRR results provide a highly reliable foundation, ensuring that the performance comparisons between ChatGPT-5.0 and Gemini 3.0 Pro are substantiated by consistent expert judgment and minimal rater bias.

To ensure full transparency and allow for a granular review of the scoring process, the detailed individual assessments from both expert raters are provided in the Appendices A–H. Specifically, six separate tables are included in Appendices C–H, each documenting the item-by-item scores for every evaluation metric used in this study.

7.1.2. Overall Performance Summary

This section presents a comprehensive comparison between ChatGPT-5.0 and Gemini 3.0 Pro based on subjective quality metrics and objective technical performance. The evaluation was conducted on 110 SQL problems spanning 11 core learning topics (Table 7), with the resulting mean scores (μ) and standard deviations (σ) summarized in Tables 12 and 13. These data provide insights into the accuracy, consistency, and practical implementability of both models as automated problem generation assistants.

Table 12. Comparison of Evaluation Scores between ChatGPT and Gemini.

Metric	ChatGPT		Gemini	
	μ	σ	μ	σ
Sensibleness	4.36	1.02	5.00	0.00
Novelty	1.27	0.47	1.86	0.32
Topicality: Concept	4.36	1.21	5.00	0.00
Topicality: Extras	2.95	0.15	4.68	0.46
Readiness: Answer	4.41	0.92	5.00	0.00

Table 13. Runnability Error Distribution Across 11 Learning Topics ($n = 110$).

Topic Index	Gemini 3.0 Pro (Errors)	ChatGPT-5.0 (Errors)
Topic 1–5	13.0	11.5
Topic 6–11	22.0	18.0
Total Errors	35.0	29.5
Mean Error per Topic	3.18 ± 2.02	2.68 ± 2.14

Based on the data presented in Tables 12 and 13, the following key insights summarize the performance of ChatGPT-5.0 and Gemini 3.0 Pro:

1. **Subjective Quality and Consistency:** Gemini 3.0 Pro demonstrates superior and highly consistent performance across nearly all subjective metrics. This is evidenced by a perfect mean score ($\mu = 5.00$) in Sensibleness, Topicality: Concept, and Readiness: Answer, with a standard deviation (σ) of 0.00. These results indicate that Gemini 3.0 Pro consistently produces logical, relevant, and classroom-ready SQL problems without quality variance between iterations. In contrast, ChatGPT-5.0 exhibits higher quality fluctuations, with σ values ranging from 0.15 to 1.21, suggesting its output is less stable compared to Gemini 3.0 Pro.

2. **Novelty and Instructional Integration:** Both models achieved their lowest scores in Novelty (Gemini 3.0 Pro: 1.86; ChatGPT-5.0: 1.27), indicating that the generated problems tend to follow common SQL exercise patterns found in existing datasets. However, Gemini 3.0 Pro is significantly more effective at integrating additional instructional elements (Topicality: Extras) with a score of 4.68, whereas ChatGPT-5.0 only reached 2.95, highlighting ChatGPT-5.0's limitations in following complex, specific instructions beyond the core concept.
3. **Technical Error Analysis (Runnability):** Despite Gemini 3.0 Pro's subjective superiority, ChatGPT-5.0 demonstrates better technical performance in the objective Runnability metric. ChatGPT-5.0 produced fewer total errors (29.5) compared to Gemini 3.0 Pro (35.0) across all 11 learning topics. On average, ChatGPT-5.0 required 2.68 corrections per topic, while Gemini 3.0 Pro required 3.18.
4. **Error Distribution per Topic:** Both models showed an increase in technical errors as the topics progressed from basic (Topics 1–5) to advanced levels (Topics 6–11). This trend reflects the increased logic complexity in later SQL topics, such as advanced joins or subqueries, which remain challenging for generative AI models. While ChatGPT-5.0 had a lower total error count, its higher standard deviation ($\sigma = 2.14$) compared to Gemini 3.0 Pro ($\sigma = 2.02$) indicates sharper error spikes in specific complex topics.

7.2. Evaluation Results for SDP

Table 14 summarizes the overall performance of the 32 participating students across SDP instances 1–5. This table shows the number of students, the average correct answer rate along with its standard deviation (SD), and the average number of answer submissions with its corresponding SD. The results indicate that the average correct answer rate was 95.20%, suggesting that students were generally able to answer the questions accurately. The average number of submissions was 1.56, implying that each student submitted their answers for a given SDP instance approximately one or two times before achieving full correctness.

Table 14. Summary of solution results across SDP instances.

Number of Students	Average Correct Answer Rate (%)	Standard Deviation	Average Submission Time	Standard Deviation
32	95.20	0.08	1.56	0.82

7.2.1. Results for Individual SDP Instances

Figure 7 illustrates the average correct answer rate and the average number of answer submissions by the 32 students for each SDP instance.

1. **Correct Answer Rate:** Figure 7 indicates that the three instances ID = 1, ID = 3, and ID = 4 show a 100% correct answer rate. An instance ID = 2 shows 96.8%, and the lowest correct answer rate is 79.20% for the instance ID = 5. The interpretation of this statistic is that students were able to answer correctly and almost perfectly for the instances ID = 1–ID = 4 but experienced difficulty with the instance ID = 5.
2. **Submission Times:** Four instances, IDs 1–4, have relatively stable submission times between 1.12 and 1.20. However, the instance ID = 5 reached 3.20. Students needed more attempts to answer correctly on the instance ID = 5. This score indicates a higher difficulty level compared to the previous instance IDs.

This result suggests a possible need for review or adjustment of the instance ID = 5, both in terms of question complexity and clarity of instructions.

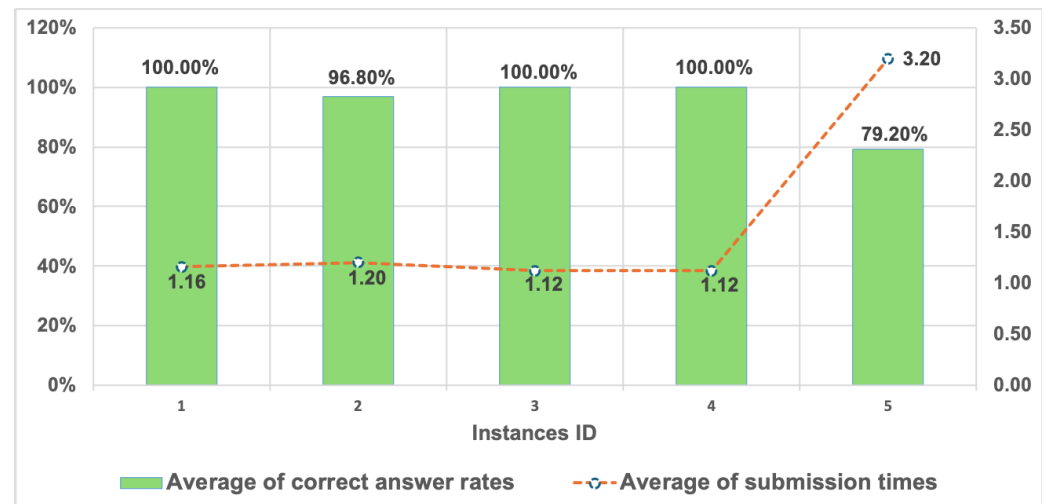


Figure 7. Average correct answer rates and submission times for each SDP instance.

7.2.2. Results for Individual Students

Figure 8 illustrates the average correct rates and the average number of answer submissions of all the SDP instances for each of the 32 first-year students.

1. **Correct Answer Rate:** Most students have a high correct answer rate, approaching 100%. The student with ID = 2 has the lowest correct answer rate at 84%. Some students such as IDs 12, 19, 21, 24, are slightly below 100%. The remaining students achieved a 100% correct answer rate. The statistics indicate that most students are able to answer correctly on most trials, meaning they have a good understanding of the material or questions.
2. **Submission Times:** The average number of submissions ranged from 1.4 to 2.0, meaning almost all students needed more than one attempt to obtain the correct answer. Four students, IDs 6, 25, 29, 32, showed the highest submission times of 2.0, indicating they needed more attempts to obtain the correct answer. Other students achieved the correct answer after 1.4 and 1.6 times. The student ID = 32, with a submission time of 2.0, was able to answer all questions correctly.

Students with high submission times will need additional support and/or guidance to complete the questions on their early attempts.

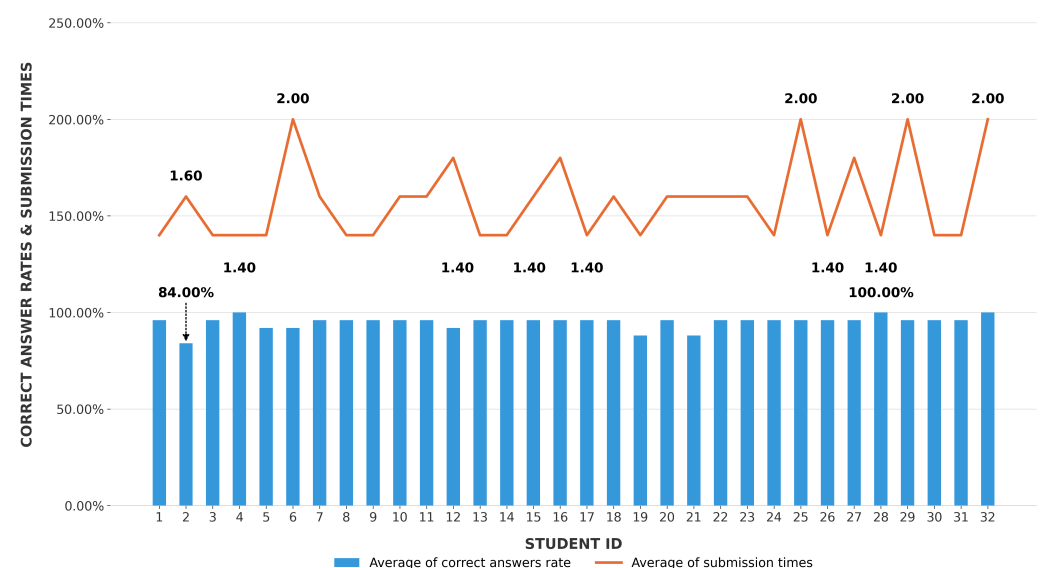


Figure 8. Average correct answer rates and submission times for each student.

7.2.3. SUS Evaluation Results

After solving the assignments, participants answered the 10 SUS questions listed in Table 8. The detailed distribution of these responses is presented in Table 15. The results indicate that the proposal was generally perceived as easy to use and well-designed. Minor findings from the usability evaluation are summarized as follows:

1. For the even-numbered statements (Q2, Q4, Q6, Q8, and Q10), the percentage of agreement responses was 0%, indicating that respondents did not perceive the system as complex or inconsistent, nor did they require technical assistance to use it.
2. For the odd-numbered statements (Q1, Q3, Q5, Q7, and Q9), the percentage of positive responses (including neutral) was 100%, suggesting that they found the answer interface intuitive, easy to learn, and felt confident using it.

Table 15. Distribution of responses for questions. All values are expressed as percentages.

Response	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
Strongly Disagree	0.00	12.50	0.00	15.63	0.00	21.88	0.00	15.63	0.00	9.38
Disagree	0.00	71.88	0.00	68.75	0.00	56.25	0.00	68.75	0.00	68.75
Neutral	9.38	15.63	18.75	15.63	9.38	21.88	9.38	15.63	31.25	21.88
Agree	56.25	0.00	46.88	0.00	46.88	0.00	46.88	0.00	56.25	0.00
Strongly Agree	34.38	0.00	34.38	0.00	43.75	0.00	43.75	0.00	12.50	0.00

Figure 9 shows that the average System Usability Scale (SUS) score was 78, which corresponds to Grade B within the Acceptable range as in Tables 9 and 10. This finding indicates that students perceived the proposed SDP as usable and effective for learning SQL programming.

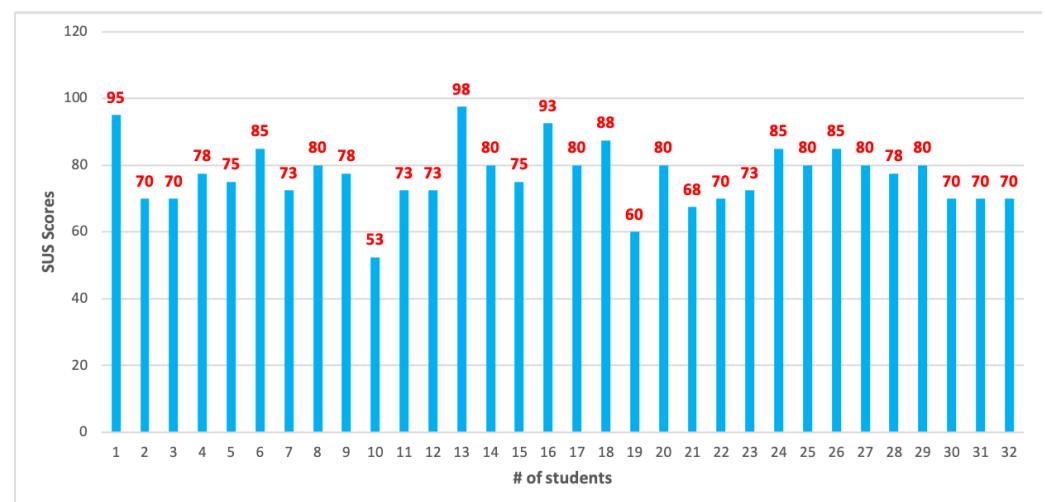


Figure 9. SUS scores.

8. Discussion

In this section, we discuss findings, implications, and limitations of the proposal.

8.1. Performance of AI Models in Pedagogical Contexts

The comparison between Gemini 3.0 Pro and ChatGPT-5.0 reveals significant insights into the suitability of LLMs for educational content generation. While both models were capable of generating SQL exercises, Gemini 3.0 Pro exhibited superior performance in terms of instructional alignment. The perfect scores in Sensibleness, Topicality, and Readiness suggest that Gemini is more reliable for teachers who require stable and ready-to-use

exercises without extensive manual editing. ChatGPT-5.0, despite its technical efficiency, occasionally required more refinement in its pedagogical logic. This finding aligns with the growing consensus that different LLMs may excel in specific niches, with Gemini showing promise in structured educational task generation.

8.2. Student Performance and the Learning Curve

The high average correct answer rate of 95.2%, coupled with a low submission average of 1.56 per question, underscores the feasibility of the SDP in reducing cognitive load for novice learners. These metrics suggest that the AI-generated descriptions provided sufficient clarity, allowing students to navigate the logic of SQL without the typical frustrations of syntax errors. By bridging the gap between conceptual intent (the ‘what’) and technical execution (the ‘how’), the SDP method facilitates better ‘scaffolding’ in programming education. This transition enables students to internalize relational logic more efficiently than traditional methods, which often lack immediate, actionable feedback.

8.3. Usability and System Acceptance

The system achieved a *System Usability Scale (SUS)* score of 78, which classifies it within the “Acceptable” and “Good” (Grade B) categories. This result is particularly significant for an educational platform, as it demonstrates that the interface successfully minimizes extraneous cognitive load. By reducing the complexity of platform navigation, the system allows students to focus their full attention on mastering SQL logic. The positive feedback from 32 students at INSTIKI indicates that the system is practical and ready to be used more widely in university classes.

8.4. Reducing Pedagogical Overhead

One of the primary goals of this research was to alleviate the burden on teachers. Traditional manual creation of SQL exercises—including schema design, data insertion, and canonical query writing—is time-consuming. The AI-assisted generator, utilizing the Spider dataset, automates this entire pipeline. This automation allows teachers to generate a vast and diverse set of exercises in a short amount of time, enabling personalized learning paths where different students can receive different sets of problems of similar difficulty levels, thereby reducing the risk of academic dishonesty.

However, the higher submission rates in some instances (e.g., ID = 5) show that question complexity and clarity still require fine-tuning. These findings confirm that SDP successfully supports independent learning while preserving teacher oversight and instructional control.

8.5. Limitations and Future Improvements

Despite the positive results, this study has several limitations. The system currently relies heavily on the quality of prompts provided to the LLMs, which can occasionally lead to logical inconsistencies or “hallucinations” when generating complex multi-table joins. Furthermore, the current assessment logic primarily uses an enhanced string-matching approach; while suitable for the current scope, it may not capture all forms of semantic equivalence in more advanced SQL queries. Additionally, this study is limited by the absence of a control group and a pre-test/post-test comparison. Moreover, the sample size of 32 students from a single institution is relatively small, which limits the statistical power and generalizability of the findings. Consequently, the high correct answer rates and positive user feedback should be interpreted as a preliminary validation of the system’s usability and task completion capability rather than a direct measurement of learning gains or long-term knowledge retention across diverse educational environments.

To address these limitations and transition from this pilot study to full-scale educational implementation, we have outlined a comprehensive three-step roadmap.

First, regarding system refinement, we will focus on enhancing the robustness of our assessment by integrating semantic analysis techniques that can evaluate query logic beyond simple textual comparison. We also aim to refine the user interface and develop a more advanced AI-driven feedback mechanism to provide helpful hints for novice learners.

Second, to address generalizability, we plan to conduct cross-semester and multi-institutional longitudinal studies. This expansion will allow us to verify the system's robustness and evaluate how this method supports long-term knowledge retention among students from diverse learning backgrounds.

Finally, we aim to formally integrate the proposed method into the SQL programming curriculum as a primary teaching tool. This will enable a comparative analysis of learning outcomes against traditional teaching methods, thereby solidifying the system's role in standard computer science education.

9. Conclusions

This paper presented the SQL Query Description Problem (SDP) as a novel exercise type within the Programming Learning Assistant System (PLAS) to enhance the self-regulated learning of SQL among novice students. To address the challenge of manual content creation for instructors, we integrated a generative AI-assisted SQL query generator utilizing the Spider dataset to automate the production of diverse and pedagogically sound exercises.

Our comparative evaluation of two leading LLMs—Gemini 3.0 Pro and ChatGPT-5.0—revealed that while ChatGPT-5.0 demonstrated higher technical runnability with fewer syntax errors, Gemini 3.0 Pro provided superior instructional alignment, achieving perfect scores in Sensibleness, Topicality, and Readiness. These findings suggest that Gemini 3.0 Pro is currently more suitable for generating ready-to-use educational materials that require minimal teacher intervention. The implementation of the system with 32 undergraduates at INSTIKI further demonstrated the feasibility of the proposal. The SDP method is easy to understand and helps students learn by providing a high average correct answer rate of 95.2% and a low submission frequency of 1.56 times per question. Moreover, the SUS score of 78 (Grade B) confirms that students found the platform accessible and easy to use.

In summary, the evaluation results from this pilot study confirm the technical feasibility and usability of the proposed system, with students demonstrating high task completion rates and positive engagement. However, to transition from this preliminary validation to a broader educational application, further improvements are necessary. Future work will focus on three key areas: refining the AI assessment logic to handle semantic variations, conducting longitudinal studies across multiple institutions to verify generalizability, and formally integrating the system into the SQL programming curriculum. These steps will allow for a comprehensive comparative analysis of learning outcomes, ultimately establishing the proposed method as a standard tool in computer science education.

Author Contributions: Conceptualization, N.W.W. and N.F.; methodology, N.W.W. and H.H.S.K.; SDP application, N.W.W. and Z.Z.; AI Assistance Exercise Generator, Z.Z. and I.N.D.K.; visualization, N.W.W. and P.S.; validation, N.W.W. and I.N.A.S.P.; writing—original draft, N.W.W.; writing—review and editing, N.F.; supervision, N.F. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: This study was conducted following the ethical guidelines of the Indonesian Institute of Business and Technology (INSTIKI). Formal approval from the Institutional Review Board was not required because the research involved normal educational practices and anonymized academic performance data, posing no risks to participants.

Informed Consent Statement: Informed consent was implied through voluntary participation in the programming exercises. All participants were informed that their anonymized performance data would be used for research purposes.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors upon request.

Acknowledgments: We would like to thank all the colleagues in the Distributing System Laboratory at Okayama University and the Department of Informatics Engineering at the Indonesian Institute of Business and Technology (INSTIKI) who participated in this study.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Keyword and Question List

Table A1. Keyword and question list.

No	Keyword	Question
1	SELECT	Write a query to select all columns from the <code>student</code> table.
2	DISTINCT	Write a query to select distinct course titles (<code>title</code>) from the <code>course</code> table, excluding courses in the Mechanical Engineering department.
3	UPDATE	Write a query to update the salary of the instructor with ID = '63395' to 100,000.
4	WHERE	Write a query to update both the salary and <code>dept_name</code> for the instructor with ID = '78699'.
5	INSERT	Write a query to insert a new student named John Doe with ID = '12345' and <code>tot_cred</code> = 30 into the <code>student</code> table.
6	DELETE	Write a query to delete the student with ID = '12345' from the <code>student</code> table.
7	INNER JOIN	Write a query to get the names of instructors and the titles of the courses they teach, using the <code>instructor</code> and <code>course</code> tables.
8	LEFT JOIN	Write a query to get the names of all students and the titles of the courses they have taken, including students who have not enrolled in any courses.
9	GROUP BY	Write a query to group instructors by department and display the total salary per department.
10	ORDER BY	Write a query to list all instructors ordered by salary in ascending order.
11	UPPER	Write a query to display all student names in uppercase letters from the <code>student</code> table.
12	LOWER	Write a query to display all instructor names in lowercase letters from the <code>instructor</code> table.
13	CONCAT	Write a query to combine <code>first_name</code> and <code>last_name</code> of students into a single column named <code>full_name</code> .
14	SUBSTRING	Write a query to extract the first 5 characters of the <code>name</code> column from the <code>student</code> table.
15	TRIM	Write a query to remove leading and trailing spaces from the <code>name</code> column in the <code>student</code> table.
16	REPLACE	Write a query to replace all occurrences of the letter "a" with "o" in the <code>name</code> column of the <code>student</code> table.
17	RPAD	Write a query to pad the <code>name</code> column of all students to a total length of 15 characters, adding spaces to the right.
18	LEFT	Write a query to extract the first 3 characters from the <code>name</code> column of the <code>student</code> table.
19	RIGHT	Write a query to extract the last 3 characters from the <code>name</code> column of the <code>student</code> table.
20	ROUND	Write a query to round the salary of all instructors to two decimal places.
21	FLOOR	Write a query to get the salary of all instructors rounded down to the nearest integer.
22	SUM	Write a query to calculate the total salary for each department using the <code>instructor</code> table.
23	COUNT	Write a query to count the number of students enrolled in each course, using the <code>student</code> and <code>takes</code> tables.
24	MOD	Write a query to find the remainder when the budget of each department is divided by 100000.
25	MIN	Write a query to find the minimum budget in the <code>department</code> table.
26	MAX	Write a query to find the highest salary in the <code>instructor</code> table.
27	CURRENT_DATE	Write a query to select all courses and add a column showing the current date.
28	YEAR	Write a query to extract the year from the <code>year</code> column in the <code>section</code> table.
29	MONTH	Write a query to extract the month from the <code>semester</code> column of the <code>section</code> table, assuming the format 'YYYY-MM'.
30	ASC	Write a query to list all students ordered by their name in ascending order.
31	DESC	Write a query to list all instructors ordered by their salary in descending order.

Appendix B. Database Schema College_2

Listing A1. University database schema.

```

CREATE TABLE classroom (
  building    VARCHAR(15),
  room_number VARCHAR(7),
  capacity    NUMERIC(4,0),
  PRIMARY KEY (building, room_number)
);

CREATE TABLE department (
  dept_name   VARCHAR(20),
  building    VARCHAR(15),
  budget      NUMERIC(12,2) CHECK (budget > 0),
  PRIMARY KEY (dept_name)
);

CREATE TABLE course (
  course_id   VARCHAR(8),
  title       VARCHAR(50),
  dept_name   VARCHAR(20) NULL,
  credits     NUMERIC(2,0) CHECK (credits > 0),
  PRIMARY KEY (course_id),
  FOREIGN KEY (dept_name) REFERENCES department (dept_name)
  ON DELETE SET NULL
);

CREATE TABLE instructor (
  ID          VARCHAR(5),
  name        VARCHAR(20) NOT NULL,
  dept_name   VARCHAR(20),
  salary      NUMERIC(8,2) CHECK (salary > 29000),
  PRIMARY KEY (ID),
  FOREIGN KEY (dept_name) REFERENCES department (dept_name)
  ON DELETE SET NULL
);

CREATE TABLE section (
  course_id   VARCHAR(8),
  sec_id      VARCHAR(8),
  semester    VARCHAR(6) CHECK (semester IN ('Fall', 'Winter', 'Spring', 'Summer')),
  year        NUMERIC(4,0) CHECK (year > 1701 AND year < 2100),
  building    VARCHAR(15),
  room_number VARCHAR(7),
  time_slot_id VARCHAR(4),
  PRIMARY KEY (course_id, sec_id, semester, year),
  FOREIGN KEY (course_id) REFERENCES course (course_id)
  ON DELETE CASCADE,
  FOREIGN KEY (building, room_number) REFERENCES classroom (building,
    room_number)
  ON DELETE SET NULL
);

CREATE TABLE teaches (
  ID          VARCHAR(5),
  course_id   VARCHAR(8),
  sec_id      VARCHAR(8),
  semester    VARCHAR(6),
  year        NUMERIC(4,0),
  PRIMARY KEY (ID, course_id, sec_id, semester, year),
  FOREIGN KEY (course_id, sec_id, semester, year) REFERENCES section (course_id
    , sec_id, semester, year)

```

```

ON DELETE CASCADE,
FOREIGN KEY (ID) REFERENCES instructor (ID)
ON DELETE CASCADE
);

CREATE TABLE student (
ID          VARCHAR(5),
name        VARCHAR(20) NOT NULL,
dept_name   VARCHAR(20),
tot_cred    NUMERIC(3,0) CHECK (tot_cred >= 0),
PRIMARY KEY (ID),
FOREIGN KEY (dept_name) REFERENCES department (dept_name)
ON DELETE SET NULL
);

CREATE TABLE takes (
ID          VARCHAR(5),
course_id   VARCHAR(8),
sec_id      VARCHAR(8),
semester    VARCHAR(6),
year        NUMERIC(4,0),
grade       VARCHAR(2),
PRIMARY KEY (ID, course_id, sec_id, semester, year),
FOREIGN KEY (course_id, sec_id, semester, year) REFERENCES section (course_id
, sec_id, semester, year)
ON DELETE CASCADE,
FOREIGN KEY (ID) REFERENCES student (ID)
ON DELETE CASCADE
);

CREATE TABLE advisor (
s_ID        VARCHAR(5),
i_ID        VARCHAR(5),
PRIMARY KEY (s_ID),
FOREIGN KEY (i_ID) REFERENCES instructor (ID)
ON DELETE SET NULL,
FOREIGN KEY (s_ID) REFERENCES student (ID)
ON DELETE CASCADE
);

CREATE TABLE time_slot (
time_slot_id VARCHAR(4),
day          VARCHAR(1),
start_hr     NUMERIC(2) CHECK (start_hr >= 0 AND start_hr < 24),
start_min    NUMERIC(2) CHECK (start_min >= 0 AND start_min < 60),
end_hr       NUMERIC(2) CHECK (end_hr >= 0 AND end_hr < 24),
end_min      NUMERIC(2) CHECK (end_min >= 0 AND end_min < 60),
PRIMARY KEY (time_slot_id, day, start_hr, start_min)
);

CREATE TABLE prereq (
course_id   VARCHAR(8),
prereq_id   VARCHAR(8),
PRIMARY KEY (course_id, prereq_id),
FOREIGN KEY (course_id) REFERENCES course (course_id)
ON DELETE CASCADE,
FOREIGN KEY (prereq_id) REFERENCES course (course_id)
);

```

Appendix C. Additional Sensibleness Evaluation Results

Table A2. Detailed sensibleness scores of AI models across learning topics, independently evaluated by two human raters. Inter-rater reliability was assessed using Cohen’s κ , and average scores are reported for completeness.

Topic	AI Model	Rater 1 Score	Rater 2 Score
Topic 1	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 2	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 3	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 4	GPT-5.0	4	4
	Gemini 3.0 Pro	5	5
Topic 5	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 6	GPT-5.0	4	4
	Gemini 3.0 Pro	5	5
Topic 7	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 8	GPT-5.0	3	4
	Gemini 3.0 Pro	5	5
Topic 9	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 10	GPT-5.0	2	2
	Gemini 3.0 Pro	5	5
Topic 11	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Inter-rater Reliability (Cohen’s κ)		$\kappa \approx 0.856209$	

Appendix D. Additional Novelty Evaluation Results

Table A3. Detailed novelty scores of AI models across learning topics, independently evaluated by two human raters. Inter-rater reliability was assessed using Cohen’s κ , and average scores are reported for completeness.

Topic Average Score	AI Model	Rater 1 Score	Rater 2 Score
Topic 1	GPT-5.0	1	1
	Gemini 3.0 Pro	1	2
Topic 2	GPT-5.0	1	1
	Gemini 3.0 Pro	1	1
Topic 3	GPT-5.0	1	1
	Gemini 3.0 Pro	2	2
Topic 4	GPT-5.0	2	2
	Gemini 3.0 Pro	2	2
Topic 5	GPT-5.0	2	2
	Gemini 3.0 Pro	2	2

Table A3. *Cont.*

Topic	Average Score	AI Model	Rater 1 Score	Rater 2 Score
Topic 6		GPT-5.0	1	1
		Gemini 3.0 Pro	2	2
Topic 7		GPT-5.0	1	1
		Gemini 3.0 Pro	2	2
Topic 8		GPT-5.0	2	2
		Gemini 3.0 Pro	2	2
Topic 9		GPT-5.0	1	1
		Gemini 3.0 Pro	2	2
Topic 10		GPT-5.0	1	1
		Gemini 3.0 Pro	2	2
Topic 11		GPT-5.0	1	1
		Gemini 3.0 Pro	2	2
Inter-rater Reliability (Cohen’s κ)			$\kappa \approx 0.9091$	

Appendix E. Additional Topicality: Concept/Keyword Evaluation Results

Table A4. Detailed topicality: concept/keyword scores of AI models across learning topics, independently evaluated by two human raters. Inter-rater reliability was assessed using Cohen's κ , and average scores are reported for completeness.

Topic	AI Model	Rater 1 Score	Rater 2 Score
Topic 1	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 2	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 3	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 4	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 5	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 6	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 7	GPT-5.0	2	2
	Gemini 3.0 Pro	5	5
Topic 8	GPT-5.0	2	2
	Gemini 3.0 Pro	5	5
Topic 9	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 10	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 11	GPT-5.0	4	4
	Gemini 3.0 Pro	5	5
Inter-rater Reliability (Cohen's κ)		$\kappa \approx 1.0$	

Appendix F. Additional Topicality: extras Evaluation Results

Table A5. Detailed Topicality: extras scores of AI models across learning topics, independently evaluated by two human raters. Inter-rater reliability was assessed using Cohen's κ , and average scores are reported for completeness.

Topic	AI Model	Rater 1 Score	Rater 2 Score
Topic 1	GPT-5.0	3	3
	Gemini 3.0 Pro	5	5
Topic 2	GPT-5.0	3	3
	Gemini 3.0 Pro	4	4
Topic 3	GPT-5.0	2	3
	Gemini 3.0 Pro	5	5
Topic 4	GPT-5.0	3	3
	Gemini 3.0 Pro	4	5
Topic 5	GPT-5.0	3	3
	Gemini 3.0 Pro	5	5
Topic 6	GPT-5.0	3	3
	Gemini 3.0 Pro	4	4
Topic 7	GPT-5.0	3	3
	Gemini 3.0 Pro	5	5
Topic 8	GPT-5.0	3	3
	Gemini 3.0 Pro	5	5
Topic 9	GPT-5.0	3	3
	Gemini 3.0 Pro	5	5
Topic 10	GPT-5.0	3	3
	Gemini 3.0 Pro	4	4
Topic 11	GPT-5.0	3	3
	Gemini 3.0 Pro	5	5
Inter-rater Reliability (Cohen's κ)		$\kappa \approx 0.85$	

Appendix G. Additional Readiness: Answers Evaluation Results

Table A6. Detailed Readiness: answers scores of AI models across learning topics, independently evaluated by two human raters. Inter-rater reliability was assessed using Cohen's κ , and average scores are reported for completeness.

Topic	AI Model	Rater 1 Score	Rater 2 Score
Topic 1	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 2	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 3	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 4	GPT-5.0	4	5
	Gemini 3.0 Pro	5	5
Topic 5	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5

Table A6. *Cont.*

Topic	AI Model	Rater 1 Score	Rater 2 Score
Topic 6	GPT-5.0	4	4
	Gemini 3.0 Pro	5	5
Topic 7	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 8	GPT-5.0	5	5
	Gemini 3.0 Pro	5	5
Topic 9	GPT-5.0	4	4
	Gemini 3.0 Pro	5	5
Topic 10	GPT-5.0	2	2
	Gemini 3.0 Pro	5	5
Topic 11	GPT-5.0	4	4
	Gemini 3.0 Pro	5	5
Inter-rater Reliability (Cohen's κ)		$\kappa \approx 0.867$	

Appendix H. Additional Readiness: Runnability Evaluation Results

Table A7. Detailed Readiness: runnability scores of AI models across learning topics, independently evaluated by two human raters. Inter-rater reliability was assessed using Cohen's κ , and average scores are reported for completeness.

Topic	AI Model	Rater 1 Score	Rater 2 Score
Topic 1	GPT-5.0	1	1
	Gemini 3.0 Pro	1	1
Topic 2	GPT-5.0	0	0
	Gemini 3.0 Pro	2	2
Topic 3	GPT-5.0	7	6
	Gemini 3.0 Pro	4	4
Topic 4	GPT-5.0	2	2
	Gemini 3.0 Pro	4	4
Topic 5	GPT-5.0	2	2
	Gemini 3.0 Pro	2	2
Topic 6	GPT-5.0	3	3
	Gemini 3.0 Pro	6	5
Topic 7	GPT-5.0	3	3
	Gemini 3.0 Pro	4	4
Topic 8	GPT-5.0	4	3
	Gemini 3.0 Pro	8	7
Topic 9	GPT-5.0	1	1
	Gemini 3.0 Pro	2	2
Topic 10	GPT-5.0	7	6
	Gemini 3.0 Pro	2	2
Topic 11	GPT-5.0	1	1
	Gemini 3.0 Pro	1	1
Inter-rater Reliability (Cohen's κ)		$\kappa \approx 0.724$	

References

1. Rahmalan, H.; Ahmad, S.S.S.; Affendey, L.S. Investigation on Designing a Fun and Interactive Learning Approach for Database Programming Subject According to Students' Preferences. *J. Phys. Conf. Ser.* **2020**, *1529*, 022076. [CrossRef]
2. Tung, S.H.; Lin, T.T.; Lin, Y.H. An Exercise Management System for Teaching Programming. *J. Softw.* **2013**, *8*, 1718–1725. [CrossRef]
3. Garner, P.; Mariani, J. Learning SQL in Steps. *Learning* **2015**, *12*, 23.
4. Migler, A.; Dekhtyar, A. Mapping the SQL Learning Process in Introductory Database Courses. In Proceedings of the 51st ACM Technical Symposium on Computer Science Education (SIGCSE '20), Portland, OR, USA, 11–14 March 2020; pp. 619–625. [CrossRef]
5. Carbone, A.; Hurst, J.; Mitchell, I.; Gunstone, D. An Exploration of Internal Factors Influencing Student Learning of Programming. In Proceedings of the Eleventh Australasian Conference on Computing Education (ACE '09), Wellington, New Zealand, 1 January 2009; Volume 95, pp. 25–33.
6. Piteira, M.; Costa, C. Learning Computer Programming: Study of Difficulties in Learning Programming. In Proceedings of the 2013 International Conference on Information Systems and Design of Communication (ISDOC '13), Lisboa, Portugal, 11–12 July 2013; pp. 75–80. [CrossRef]
7. Ahadi, A.; Behbood, V.; Vihavainen, A.; Prior, J.; Lister, R. Students' Syntactic Mistakes in Writing Seven Different Types of SQL Queries and Its Application to Predicting Students' Success. In Proceedings of the 47th ACM Technical Symposium on Computing Science Education (SIGCSE '16), Memphis, TN, USA, 2–5 March 2016; pp. 401–406. [CrossRef]
8. Taipalus, T. The Effects of Database Complexity on SQL Query Formulation. *J. Syst. Softw.* **2020**, *170*, 110576. [CrossRef]
9. Taipalus, T.; Perälä, P. What to Expect and What to Focus on in SQL Query Teaching. In Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE '19), Minneapolis, MN, USA, 27 February 2019–2 March 2019; pp. 198–203. [CrossRef]
10. Aerts, W.; Fletcher, G.; Miedema, D. A Feasibility Study on Automated SQL Exercise Generation with ChatGPT-3.5. In Proceedings of the 3rd International Workshop on Data Systems Education: Bridging Education Practice with Education Research (DataEd '24), Santiago, Chile, 9 June 2024; pp. 13–19. [CrossRef]
11. Yu, T.; Zhang, R.; Yang, K.; Yasunaga, M.; Wang, D.; Li, Z.; Ma, J.; Li, I.; Yao, Q.; Roman, S.; et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP 2018), Brussels, Belgium, 31 October–4 November 2018; pp. 3911–3921. [CrossRef]
12. OpenAI. Introducing GPT-5. 2025. Available online: <https://openai.com/index/introducing-gpt-5/> (accessed on 5 November 2025).
13. Google DeepMind. Gemini 3.0 Pro. 2025. Available online: <https://deepmind.google/models/gemini/pro/> (accessed on 4 December 2025).
14. Mitrovic, A. Learning SQL with a Computerized Tutor. In Proceedings of the Twenty-Ninth SIGCSE Technical Symposium on Computer Science Education (SIGCSE '98), Atlanta, GA, USA, 26 February–1 March 1998; pp. 307–311. [CrossRef]
15. Akhuseyinoglu, K.; Hardt, R.; Barria-Pineda, J.; Brusilovsky, P.; Pollari-Malmi, K.; Sirkiä, T.; Malmi, L. A Study of Worked Examples for SQL Programming. In Proceedings of the Proceedings of the 27th ACM Conference on Innovation and Technology in Computer Science Education—Vol. 1 (ITiCSE 2022), Dublin, Ireland, 8–13 July 2022; pp. 82–88. [CrossRef]
16. Luckin, R.; Holmes, W. *Intelligence Unleashed: An Argument for AI in Education*; Pearson: London, UK, 2016.
17. Chen, L.; Chen, P.; Lin, Z. Artificial Intelligence in Education: A Review. *IEEE Access* **2020**, *8*, 75264–75278. [CrossRef]
18. Baidoo-Anu, D.; Ansah, L.O. Education in the Era of Generative Artificial Intelligence (AI): Understanding the Potential Benefits of ChatGPT in Promoting Teaching and Learning. *J. AI* **2023**, *7*, 52–62. [CrossRef]
19. Choi, W.C.; Chang, C.I. ChatGPT-5 in Education: New Capabilities and Opportunities for Teaching and Learning. *Preprints* **2025**, 1–16. [CrossRef]
20. Pornphol, P.; Chittayasothorn, S. Verification of Relational Database Languages Codes Generated by ChatGPT. In Proceedings of the Proceedings of the 2023 4th Asia Service Sciences and Software Engineering Conference (ASSE '23), Aizu-Wakamatsu, Japan, 27–29 October 2023; ACM: New York, NY, USA, 2023; pp. 17–22. [CrossRef]
21. Li, Z.; Xie, T. Using LLM to Select the Right SQL Query from Candidates. *arXiv* **2024**, arXiv:2401.02115. [CrossRef]
22. Gupta, M.S.; Kumar, N.; Rao, V. AI and Teacher Productivity: A Quantitative Analysis of Time-Saving and Workload Reduction in Education. In Proceedings of the Conference on Advancing Synergies in Science, Engineering, and Management (ASEM 2024), Virginia Beach, VA, USA, 6–9 November 2024; pp. 97–104.
23. Hashem, R.; Ali, N.; El Zein, F.; Fidalgo, P.; Abu Khurma, O. AI to the Rescue: Exploring the Potential of ChatGPT as a Teacher Ally for Workload Relief and Burnout Prevention. *Res. Pract. Technol. Enhanc. Learn.* **2024**, *19*, 1–26. [CrossRef]
24. Mason, R.; Seton, C.; Cooper, G. Applying Cognitive Load Theory to the Redesign of a Conventional Database Systems Course. *Comput. Sci. Educ.* **2016**, *26*, 68–87. [CrossRef]

25. Yousoof, M.; Sapiyan, M.; Kamaluddin, K. Reducing Cognitive Load in Learning Computer Programming. *World Acad. Sci. Eng. Technol. Int. J. Comput. Inf. Eng.* **2007**, *1*, 3751–3754.
26. Ma, L.; Zhang, X.; Wang, Z.; Luo, H. Designing Effective Instructional Feedback Using a Diagnostic and Visualization System: Evidence from a High School Biology Class. *Systems* **2023**, *11*, 364. [\[CrossRef\]](#)
27. Sudo, K.; Watanuki, S.; Matsuoka, H.; Otake, E.; Yatomi, Y.; Nagaoka, N.; Iino, K. Effects of the Project on Enhancement of Teaching Skills in Gerontic Nursing Practice of Indonesian Nursing Lecturer and Clinical Nurse Preceptor. *Glob. Health Med.* **2023**, *5*, 285–293. [\[CrossRef\]](#) [\[PubMed\]](#)
28. Harvey, L., III; Aggarwal, A. Exploring the Effect of Quiz and Homework Submission Times on Students' Performance in an Introductory Programming Course in a Flipped Classroom Environment. In Proceedings of the 2021 ASEE Virtual Annual Conference, Virtual, 26–29 July 2021; pp. 1–16. [\[CrossRef\]](#)
29. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention Is All You Need. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS 2017), Long Beach, CA, USA, 4–9 December 2017; Volume 30.
30. Rosca, C.M.; Stancu, A. Quality Assessment of GPT-3.5 and Gemini 1.0 Pro for SQL Syntax. *Comput. Stand. Interfaces* **2026**, *95*, 104041. [\[CrossRef\]](#)
31. Miedema, D.; Taipalus, T.; Aivaloglou, E. Students' Perceptions on Engaging Database Domains and Structures. In Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE 2023), Toronto, ON, Canada, 15–18 March 2023; Volume 1, pp. 122–128. [\[CrossRef\]](#)
32. Funabiki, N.; Tana, Z.; Zaw, K.K.; Ishihara, N.; Kao, W.C. A Graph-Based Blank Element Selection Algorithm for Fill-in-Blank Problems in Java Programming Learning Assistant System. *IAENG Int. J. Comput. Sci.* **2017**, *44*, 247–260.
33. Aung, S.T.; Funabiki, N.; Syaifudin, Y.W.; Kyaw, H.H.S.; Aung, S.L.; Dim, N.K.; Kao, W.C. A Proposal of Grammar-Concept Understanding Problem in Java Programming Learning Assistant System. *J. Adv. Inf. Technol. (JAIT)* **2021**, *12*, 342–350. [\[CrossRef\]](#)
34. Lu, X.; Funabiki, N.; Kyaw, H.H.S.; Htet, E.E.; Aung, S.L.; Dim, N.K. Value Trace Problems for Code Reading Study in C Programming. *Adv. Sci. Technol. Eng. Syst. J. (ASTESJ)* **2022**, *7*, 14–26. [\[CrossRef\]](#)
35. Institut Bisnis dan Teknologi Indonesia (INSTIKI). Modul Praktikum Basis Data Lanjut. 2022. Available online: <https://instiki.ac.id/wp-content/uploads/2022/02/Modul-Praktikum-Basis-Data-Lanjut.pdf> (accessed on 28 August 2025).
36. W3Schools. SQL Tutorial. 2025. Available online: <https://www.w3schools.com/sql/> (accessed on 28 October 2025).
37. Unal, E.; Çakir, H. Students' Views about the Problem-Based Collaborative Learning Environment Supported by Dynamic Web Technologies. *Malays. Online J. Educ. Technol.* **2017**, *5*, 1–19.
38. Lu, X.; Funabiki, N.; Naing, I.; Kyaw, H.H.S.; Ueda, K. A Proposal of Two Types of Exercise Problems for TCP/IP Programming Learning by C Language. In Proceedings of the IEICE Technical Report, Tokyo, Japan, 31 October 2023; NS2022-236; pp. 396–401.
39. Sarsa, S.; Denny, P.; Hellas, A.; Leinonen, J. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In Proceedings of the 2022 ACM Conference on International Computing Education Research (ICER '22), Lugano, Switzerland, 7–11 August 2022; Volume 1, pp. 27–43. [\[CrossRef\]](#)
40. Gwet, K.L. *Handbook of Inter-Rater Reliability*; STATAXIS Publishing Company: Gaithersburg, MD, USA, 2001.
41. Cohen, J. A Coefficient of Agreement for Nominal Scales. *Educ. Psychol. Meas.* **1960**, *20*, 37–46. [\[CrossRef\]](#)
42. Landis, J.R.; Koch, G.G. The Measurement of Observer Agreement for Categorical Data. *Biometrics* **1977**, *33*, 159–174. [\[CrossRef\]](#) [\[PubMed\]](#)
43. Tullis, T.S.; Stetson, J.N. A Comparison of Questionnaires for Assessing Website Usability. In Proceedings of the Usability Professional Association Conference, Minneapolis, MN, USA, 7–11 June 2004; Volume 1, pp. 1–12.
44. Brooke, J. SUS: A Quick and Dirty Usability Scale. In *Usability Evaluation in Industry*; Jordan, P.W., Thomas, B., Weerdmeester, B.A., McClelland, A.L., Eds.; Taylor & Francis: London, UK, 1996; pp. 189–194.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.